

Lab 8: JPA Table Relationships

วิชา: CP353002 Principles of Software Design

เรื่อง: การจัดการความสัมพันธ์ของตาราง (1:1 และ 1:N) ด้วย Spring Data JPA และ PostgreSQL

โดย นายอนันต์เอกก์ใหญ่พงศกร 673380430-9 sec3

ส่วนที่ 1: หลักการออกแบบ (Software Design & Principles)

1.1 การประยุกต์ใช้ SOLID Principles กับ Entity

- Single Responsibility Principle (SRP): แยกหน้าที่ ProductDetail ออกจาก Product เพื่อให้ Product ทำหน้าที่เก็บเฉพาะข้อมูลพื้นฐานของสินค้า ส่วนข้อมูลรายละเอียดเชิงลึก (เช่น ระยะเวลาประกัน, น้ำหนัก, ขนาด) ถูกดูแลโดย ProductDetail ช่วยลดความซับซ้อนของคลาส
- Open/Closed Principle (OCP): การเพิ่มหน้าที่ Review เข้าสู่ระบบ ทำได้โดยการสร้างคลาสใหม่ พร้อมผูกความสัมพันธ์แบบ @ManyToOne กลับมาที่ Product โดยไม่ต้องแก้ไขโครงสร้างฟิลด์เดิมของ Product
- Dependency Inversion Principle (DIP): คลาสในชั้น Service และ Controller พึ่งพาผ่าน Abstraction (ProductRepository, DiscountStrategy) แทนการยึดติดกับ Concrete Implementation

1.2 ความแตกต่างระหว่าง 1:1 (One-to-One) และ 1:N (One-to-Many)

- 1:1 (One-to-One):
 - o นิยาม: ข้อมูล 1 รายการในตาราง A มีความสัมพันธ์กับข้อมูลในตาราง B เพียง 1 รายการเท่านั้น (Product ↔ ProductDetail)
 - o กรณีที่ควรใช้: ใช้เมื่อต้องการแยกกลุ่มข้อมูลที่มีขนาดใหญ่ ข้อมูลที่เป็นตัวเลือกเสริม (Optional) หรือข้อมูลที่มีอัตราการเรียกใช้งานต่างกัน ออกจากตารางหลัก เพื่อเพิ่มประสิทธิภาพการสอบถามข้อมูล (Query Performance)
- 1:N (One-to-Many):
 - o นิยาม: ข้อมูล 1 รายการในตาราง A สามารถมีความสัมพันธ์กับข้อมูลในตาราง B ได้หลายรายการ (Product 1 ชิ้น มีได้หลาย Review)
 - o กรณีที่ควรใช้: ใช้เมื่อข้อมูลฝั่งลูกมีจำนวนไม่แน่นอน และต้องอ้างอิงกลับมาหาเอนทิตีหลักเสมอ โดยฝั่ง Many (ตารางลูก) จะเป็นฝั่งที่ถือ Foreign Key (product_id)

1.3 Strategy Pattern สำหรับคำนวณส่วนลด ระบบนำ Strategy Pattern มาใช้จัดการอัลกอริทึม ส่วนลดราคาสินค้า

ประกอบด้วย:

- DiscountStrategy (Interface): กำหนดเมธอดมาตรฐาน calculateDiscount(price)
- Concrete Strategies: NoDiscountStrategy (ราคาปกติ), MemberDiscountStrategy (ลด 10%), SeasonalSaleStrategy (ลด 20%)
- DiscountContext: คลาสสำหรับเลือกอัลกอริทึมที่เหมาะสมตามค่า discountType ช่วยให้เพิ่มโปรโมชั่นใหม่ๆ ในอนาคตได้ทันทีโดยไม่ต้องแก้ไขโค้ดคำนวณเดิม

1.4 ลำดับการทำงาน (Execution Flow: HTTP Request -> Database)

1. HTTP Request: ผู้ใช้ส่ง Request ผ่าน Browser (เช่น กดบันทึกสินค้าใหม่)
2. Controller Layer: ProductController รับ Request และ Bind ข้อมูลเข้ากับวัตถุ Product (พร้อม ProductDetail และ Review)
3. Service Layer: ProductController ส่งวัตถุไปให้ ProductService เพื่อประมวลผล Business Logic
4. Strategy Execution: ProductService เรียก DiscountContext คำนวณราคาสุทธิ (discountedPrice)
5. Database Persistence: ProductService เรียก ProductRepository.save() เพื่อให้ Spring Data JPA บันทึกข้อมูลลงฐานข้อมูล PostgreSQL ผ่าน Cascade Operations
6. UI Rendering: Controller ส่งคำสั่ง Redirect กลับไปหน้า /products พร้อมแสดงข้อความแจ้งเตือนผ่าน Thymeleaf

ส่วนที่ 2: Code + คำอธิบาย (Implementation Details)

2.1 โครงสร้าง Entity ทั้ง 3 ตัว และการใช้ Annotations

- Product.java (Main Entity - ฝั่ง One)
 - o @Entity / @Table(name = "products"): กำหนดให้คลาสเป็น JPA Entity และแมปกับตาราง products
 - o @Id / @GeneratedValue(strategy = GenerationType.IDENTITY): กำหนด Primary Key แบบ Auto-increment
 - o @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true): สร้างความสัมพันธ์ 1:1 กับ ProductDetail โดยกำหนดให้จัดการข้อมูลลูกตามตารางแม่แบบอัตโนมัติ
 - o @JoinColumn(name = "detail_id"): กำหนดให้ตาราง products เป็นฝั่งถือ Foreign Key ชื่อ detail_id
 - o @OneToMany(mappedBy = "product", cascade = CascadeType.ALL): สร้างความสัมพันธ์ 1:N กับ Review โดยอ้างอิงฟิลด์ product ในคลาส Review

- ProductDetail.java (1:1 Dependent Entity)
 - o @Entity / @Table(name = "product_details"): แมปกับตาราง product_details
 - o @Column(columnDefinition = "TEXT"): กำหนดให้ฟิลด์ description เก็บข้อความขนาดใหญ่ใน PostgreSQL
- Review.java (1:N Many Entity - ฝั่งถือ FK)
 - o @Entity / @Table(name = "reviews"): แมปกับตาราง reviews
 - o @ManyToOne: กำหนดความสัมพันธ์ N:1 กลับไปยัง Product
 - o @JoinColumn(name = "product_id"): กำหนดคอลัมน์ Foreign Key product_id ในตาราง reviews เพื่อใช้อ้างอิงกลับไปยังตาราง products

2.2 การประยุกต์ใช้ Constructor Injection ใน Service และ Controller

ระบบเลือกใช้ Constructor Injection แทนการใช้ @Autowired บน Field โดยตรง เพื่อปฏิบัติตามหลัก Dependency Inversion (DIP) และช่วยให้ Class เป็น Immutable (กำหนดตัวแปรเป็น final ได้):

- ProductService.java Injection:

```
Java
@Service
public class ProductService {
    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    public ProductService(ProductRepository productRepository,
DiscountContext discountContext) {
        this.productRepository = productRepository;
        this.discountContext = discountContext;
    }
}
```

- ProductController.java Injection:

Java

```
@Controller
```

```
@RequestMapping("/products")
```

```
public class ProductController {  
    private final ProductService productService;
```

```
    public ProductController(ProductService productService) {  
        this.productService = productService;
```

```
    }
```

```
}
```

ส่วนที่ 3: ภาพหน้าจอการทำงาน (Application Screenshots)

3.1 หน้าจอการเพิ่มสินค้าใหม่ (Create Form)

+ เพิ่มสินค้าใหม่

ข้อมูลสินค้า (Product)

ชื่อสินค้า: iPhone 15 Pro (673380430-9 sec3) หมวดหมู่: Electronics

ยี่ห้อ: Apple จำนวนในสต็อก: 10

ราคารวม (บาท): 45000 ส่วนลดสมาชิก (10%)

รายละเอียดสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า: Phone

ระยะเวลาประกัน: 1 Year น้ำหนัก (กิโลกรัม): 1.67

ขนาด (กว้าง x ยาว x สูง): 7.06 x 14.66 x 0.82 cm ประเทศที่ผลิต: Thailand

รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

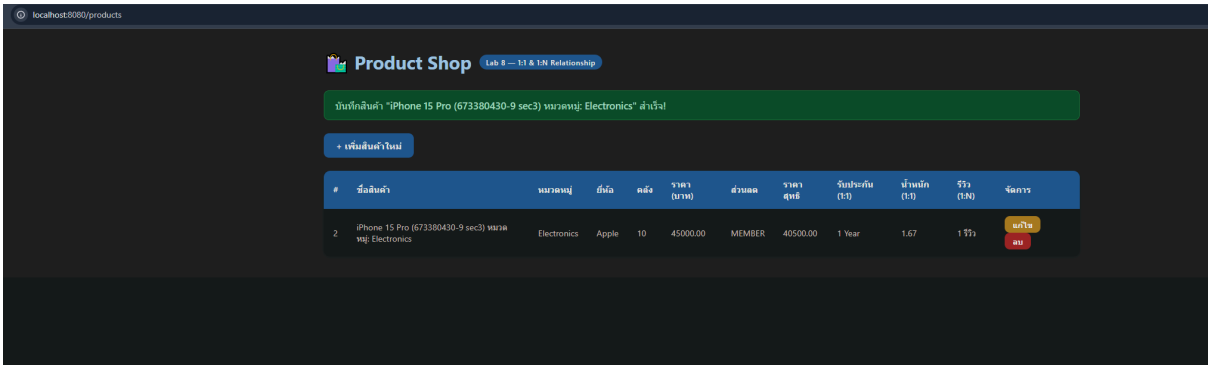
ชื่อผู้รีวิว: อันันต์เอกก์ โงส์ คะแนน (1-5): 5

ความเห็น: wow

บันทึกสินค้า ยกเลิก

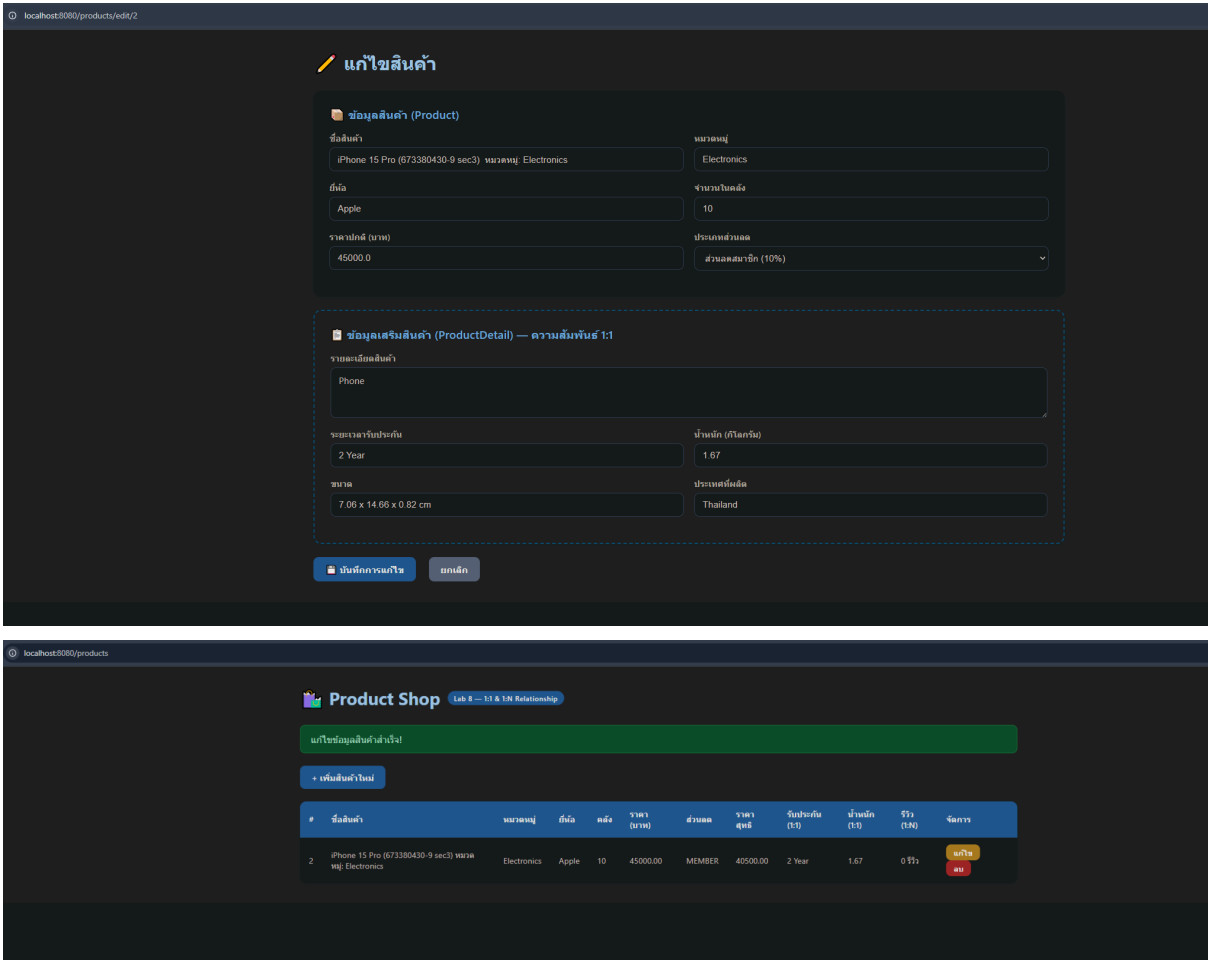
รูปที่ 3.1: หน้าฟอร์มเพิ่มสินค้าใหม่ (/products/add) กรอกข้อมูล Product, ProductDetail (1:1) และ Review (1:N) โดยระบุชื่อสินค้าเป็น iPhone 15 Pro (673380430-9 SEC 3)

3.2 หน้าจอแสดงรายการสินค้าในตาราง (Read)



รูปที่ 3.2: หน้าแสดงรายการสินค้าทั้งหมด (/products) แสดงข้อมูลราคาสุทธิที่คำนวณจาก Strategy Pattern พร้อมข้อมูลรับประกัน (1:1) และจำนวนรีวิว (1:N)

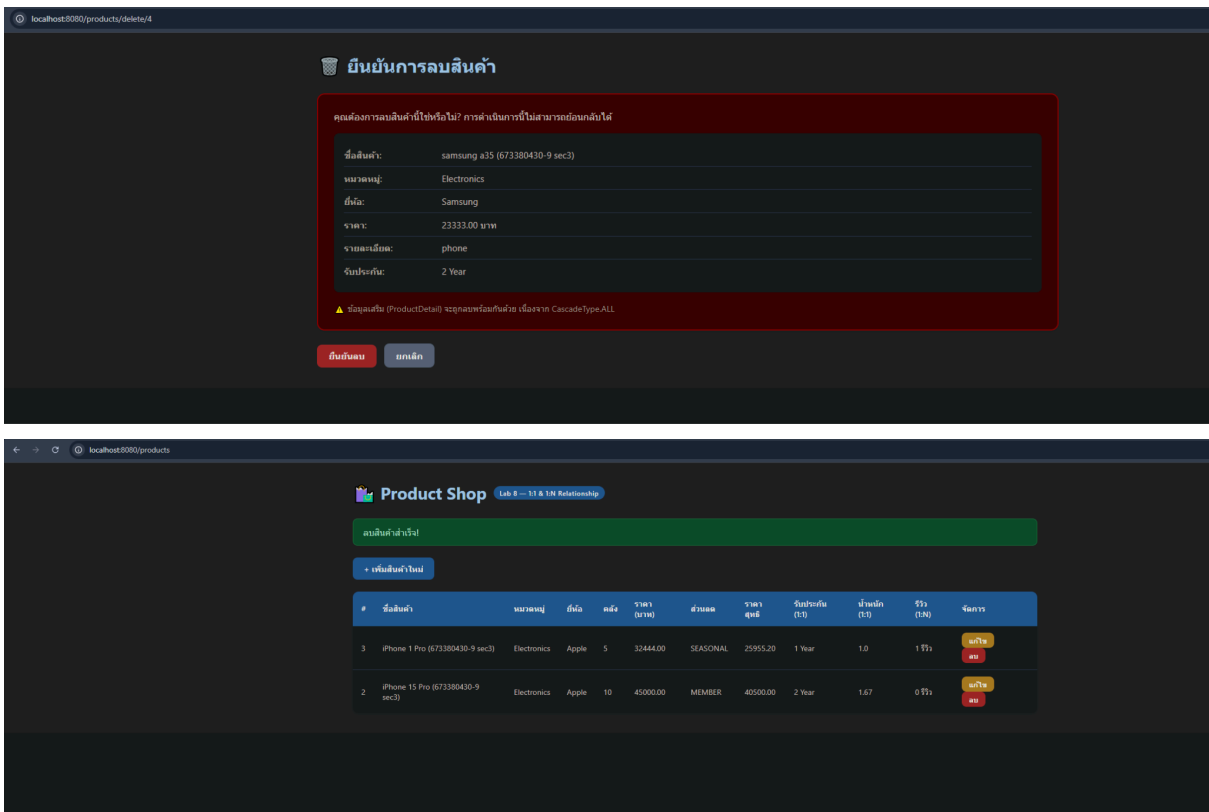
3.3 หน้าจอแก้ไขข้อมูลสินค้า (Update Form)



รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลสินค้า (/products/edit/{id})

แสดงข้อมูลเดิมจากฐานข้อมูลที่ถูกดึงมาใส่อย่างถูกต้อง

3.4 หน้าจอยืนยันการลบและการลบข้อมูล (Delete)



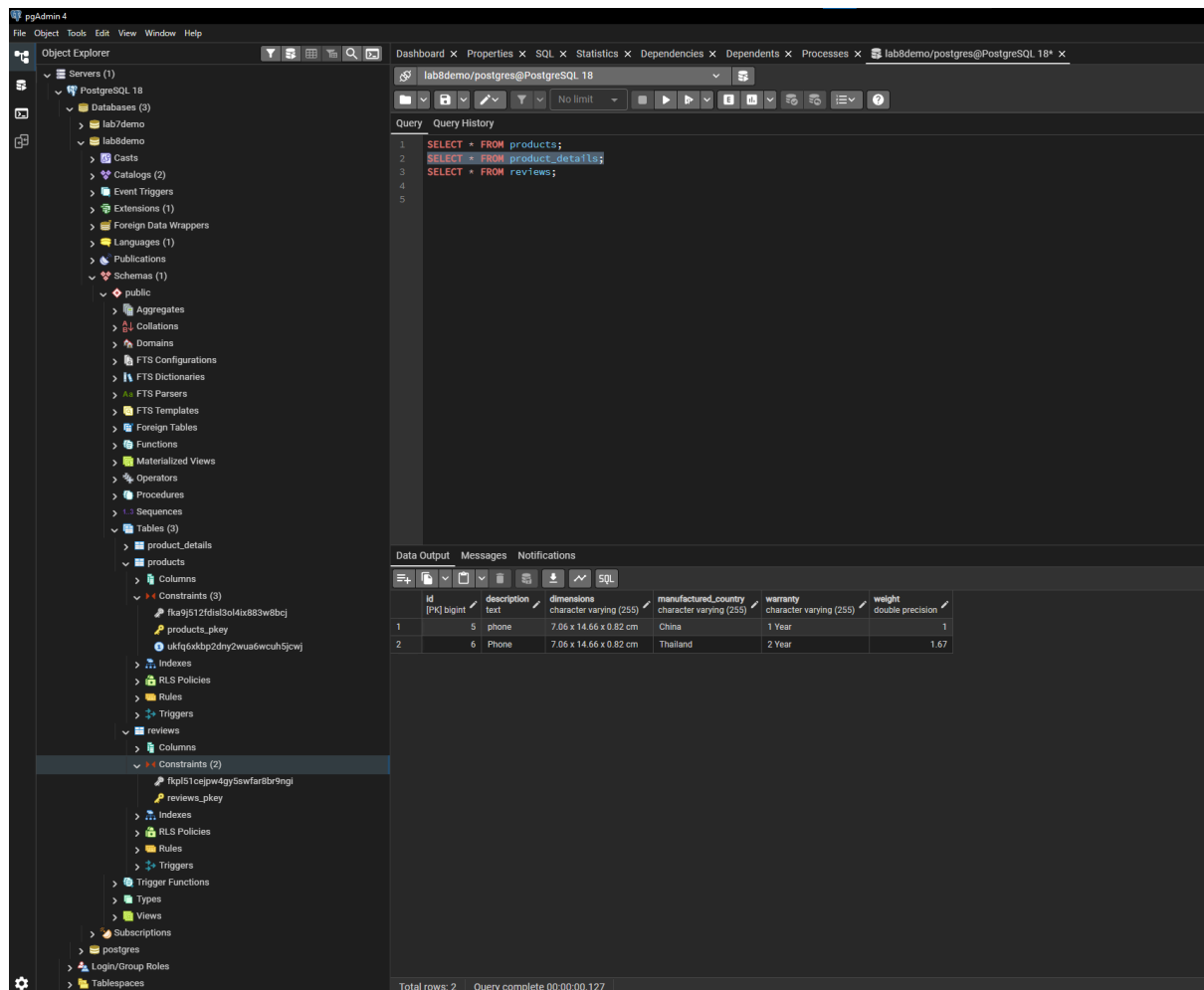
รูปที่ 3.4: หน้าจอยืนยันการลบสินค้า (/products/delete/{id}) และหน้าตารางหลังทำการลบรายการสำเร็จ แสดงแจ้งเตือนสีเขียว

3.5 หน้าจอตรวจสอบข้อมูลใน pgAdmin (Database)

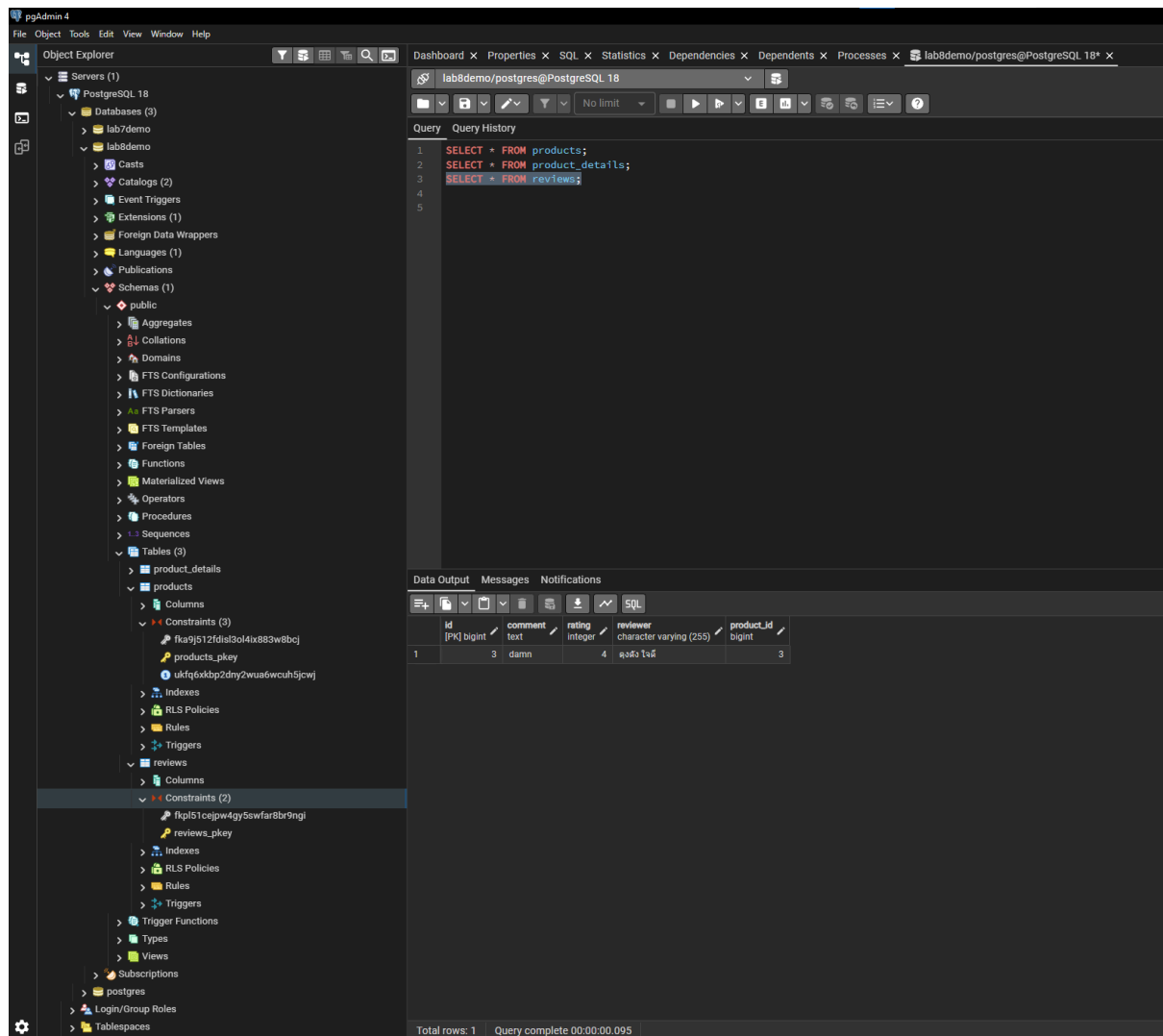
The screenshot displays the pgAdmin 4 interface. On the left, the 'Object Explorer' shows the database structure. The 'public' schema is expanded, showing the 'products' table. The 'Columns' tab for 'products' is selected, showing the 'detail_id' column with a foreign key icon. The 'Data Output' pane at the bottom right shows the results of a query, displaying two rows of data for the 'products' table.

id	brand	category	discount_type	name	price	stock	detail_id
1	Apple	Electronics	SEASONAL	iPhone 1 Pro (573380430-9 sec3...	32444	5	5
2	Apple	Electronics	MEMBER	iPhone 15 Pro (673380430-9 sec...	45000	10	6

รูปที่ 3.5.1: แสดงตารางproducts ใน pgAdmin พร้อมตรวจสอบคอลัมน์ Foreign Key (detail_id และ product_id)



รูปที่ 3.5.2: แสดงตาราง product_details ใน pgAdmin พร้อมตรวจสอบคอลัมน์ Foreign Key (detail_id และ product_id)



รูปที่ 3.5.3: แสดงตาราง reviews ใน pgAdmin พร้อมตรวจสอบคอลัมน์ Foreign Key (detail_id และ product_id)